DEADLINEGUARD

Deadline-Aware Assignment Submission and Server Overload Protection System

Technical Project Documentation

Full-stack web application | React frontend | Python FastAPI backend | SQLite | Chunked uploads

Document date: 27 October 2026

Document basis: Latest user-provided backend files: main(1).py, rate_limiter(1).py, database(1).py, models(1).py, upload_service(1).py, auth(1).py, deadline_service(1).py, presentation.py, seed.py, errors.py and requirements.txt.

Scope note: This documents observed source code, intended system behavior, known limitations and a proposed validation plan. It does not claim that a live deployment or performance improvement was verified.

1. Executive Summary

DeadlineGuard is a college assignment submission platform built to address a specific problem: last-minute bursts of uploads combined with network interruptions. The project provides faculty assignment creation, student uploading, server-side deadline checks, resumable chunk transfers, admission controls, integrity checks and submission receipts.

The central design separates **new upload-session admission** from **continuation of an existing authorized upload**. This matters because one document generates multiple HTTP requests. A naive per-request rate limit can penalize a legitimate multipart upload.

2. Problem Statement and User Benefit

Problem: As a deadline approaches, many students may attempt to upload documents concurrently. High traffic, repeat clicks, connection failures and timeouts can interfere with completion. A partly uploaded file must not be mistaken for a finalized submission.

Stakeholder	Practical benefit
Student	Resume only missing upload chunks after interruption; obtain a recorded receipt after verification.
Faculty	Define assignment deadline and recovery period; inspect finished submissions and upload histories.
System administrator	Use bounded transfer concurrency, rate limits, and audit records to control and examine load.

Important fairness qualification: Creating an upload session before a deadline is not the same as finishing submission. A grace-recovered submission must be labeled separately. A pre-deadline SHA-256 hash commitment does not prove that the student possessed the full file before the deadline.

3. Objectives

• Provide a functioning student/faculty assignment workflow.

• Preserve accepted upload chunks and permit eligible recovery.

• Enforce deadlines with server-controlled UTC time.

• Control session creation separately from continuation traffic.

• Prevent duplicate chunks and more than one final submission per student per assignment.

• Generate receipts only after file-size, SHA-256 and PDF checks.

• Compare baseline and proposed policies through a controlled experiment.

4. Research Motivation and Gap

Research reference: Xuan Chen and John Heidemann, “Flash Crowd Mitigation via Adaptive Admission Control Based on Application-Level Observations,” ACM Transactions on Internet Technology, vol. 5, no. 3, 2005. DOI: 10.1145/1084772.1084776.

This paper examines admission control for sudden traffic surges. DeadlineGuard adapts the broader overload problem to deadline-bound academic uploads. The paper did not establish that session-aware upload recovery is novel, and the project should not claim first invention of rate limiting, resumable transfer or upload sessions.

Proposed evaluation question: Under the same test workload, do separate session-admission and continuation limits combined with chunk recovery improve completed uploads or retransmission overhead compared with a common request budget? This remains to be established experimentally.

5. Architecture

```
React UI -> HTTP /api endpoints -> FastAPI (main.py)
  | auth.py / deadline_service.py / rate_limiter.py
  | upload_service.py (chunks, hash, finalization)
  | SQLAlchemy -> SQLite metadata
  | disk storage -> temporary chunks and final PDFs
  | presentation.py -> JSON for React
```

Architecture choice: A single-process local FastAPI prototype with SQLite and local disk files keeps the system understandable. The concurrency lock, semaphore, session dictionary and rate-limit counters are process-local, so they are not a production multi-server solution.

6. Technology Stack

Technology	Role
React / JavaScript	Student and faculty screens; uploads by HTTP requests.
FastAPI / Pydantic	Routes, authentication dependencies and request validation.
SQLAlchemy / SQLite	Relational records for users, assignments, upload sessions and receipts.
hashlib SHA-256	Chunk and final-file integrity comparisons.
pypdf	Checks PDF readability, encryption status and page count.
Python threading / asyncio	Locks for local mutations; semaphore for active transfer slots; asynchronous request streaming.

7. Database Design

Table	Key fields and purpose
User	id, username, password_hash, role; stores student and faculty accounts.
Assignment	id, faculty_id, deadline_at, recovery_minutes, max_file_size.
UploadSession	id, student_id, assignment_id, expected_file_hash, chunk_size, total_chunks, status, created_at, expires_at.
UploadChunk	upload_session_id, chunk_index, chunk_hash, stored_path; unique (session,index).
Submission	student_id, assignment_id, upload_session_id, final_file_hash, submitted_at, status; unique (student,assignment).
AuditLog	upload_session_id, event, detail, timestamp.

The database records metadata and finalization events; PDFs are stored as files, not as database BLOBs. Foreign keys connect the records. SQLite uniqueness checks are the final line of defense against duplicate finalized records.

8. Core Backend Modules and Functions

8.1 deadline_service.py

Function	Explanation
to_naive_utc(dt)	Converts a timezone-aware input to UTC without tzinfo for consistent SQLite date storage.
session_expiry(assignment)	Adds the configured recovery period to the assignment deadline.
check_new_session(assignment, now)	Returns whether a NEW session may start before the deadline.
check_existing_session(session, assignment, now)	Returns whether an EXISTING session can continue before expiry.
classify_submission(assignment, now)	Labels finalized submission on-time or recovered-within-grace.

8.2 rate_limiter.py

Function	Explanation
SlidingWindowLimiter.__init__	Stores the maximum request count, window length, timestamp deques and lock.
SlidingWindowLimiter.allow	Acquires the lock and checks admission.
SlidingWindowLimiter._allow_locked	Removes old timestamps; admits or rejects; returns retry delay.
check_rate_limit(user_id, kind)	Chooses naive shared budget or session-aware new-session/other budget.
reset()	Clears counters for experiments; currently not lock-protected.

Current source settings: proposed mode has 3 new session requests/minute and 240 continuation requests/minute per user; the naive mode has 30 total requests/minute per user. These are chosen prototype settings, not calibrated performance limits.

8.3 auth.py

Function	Explanation
hash_password	PBKDF2-HMAC-SHA256 with random salt; stores salt and hash.
verify_password	Recomputes and compares hashes using compare_digest.
create_login_token	Creates random bearer-like cookie token in the process-local SESSIONS dictionary.
get_current_user	Reads cookie, maps token to account, retrieves user from DB.
require_faculty	Rejects users without faculty role.

8.4 database.py, models.py, seed.py, errors.py, presentation.py

Function	Explanation
configure_sqlite	Enables foreign keys and WAL on SQLite connections.
get_db	Yields one SQLAlchemy session and closes it after request.

Function	Explanation
utcnow	Returns current server time in naive UTC format.
seed_demo	Creates seeded accounts and assignments if absent.
AppError.__init__	Stores status, message and structured extra information.
timestamp	Converts stored UTC datetime to Unix seconds for React.
upload_view	Adapts upload record and missing-chunk data to UI response fields.
receipt_view	Adapts final submission to a UI receipt.

8.5 upload_service.py - upload lifecycle

Function	Explanation
write_audit	Appends an audit event to the database transaction.
get_owned_session	Checks that the requested session belongs to the logged-in student.
create_upload_session	Checks role, deadline, assignment, file metadata, duplicates and active-session cap; creates metadata row.
receive_chunk	Checks owner, status, deadline, index and exact byte count; hashes and stores chunk; handles duplicate re-send.
received_indexes	Returns indexes of all stored chunks in ascending order.
chunk_status	Returns chunk acknowledgment and count information.
get_upload_status	Computes received and missing indexes to support resume.
find_active_session	Locates existing incomplete upload after refresh/login.
cancel_upload	Cancels active session and removes temporary folder.
submission_to_dict	Constructs a final submission response.
fail_session	Marks failed upload, writes audit and raises an error.
finalize_submission	Assembles chunks, checks hash/size/PDF/deadline, creates unique submission and receipt.

9. FastAPI Routes in main.py

Route handler	Purpose
lifespan	Seeds demo data when application starts.
local_browser_safety	Local origin allowlist for mutations; adds no-sniff/no-store headers.
health; handle_app_error	Health JSON and consistent conversion of AppError into HTTP responses.
enforce_limit; iso; assignment_to_dict	Shared routing helpers for limits and serialization.
login; logout; me	Cookie login, logout and identity.

Route handler	Purpose
create_assignment; list_assignments	Faculty creates assignments; authenticated user lists assignments.
start_upload	Validates new session request; uses new-session admission budget.
upload_chunk; save_chunk	Streams chunk with 30s timeout, four active transfer slots; stores under local lock.
upload_status; active_upload	Returns missing chunks or currently active upload.
finalize; cancel	Completes upload or cancels a session.
my_submissions; download	Student list and authorized PDF download.
faculty_submissions; audit	Faculty review and audit trail.
faculty_analytics; faculty_sessions	Counts final submissions and upload-session status.
session_for_frontend; submissions_for_frontend	Compatibility endpoints expected by existing React interface.
authorized_submission; receipt_download	Ownership check; JSON receipt download.
demo_file; comparison	Serves bundled demo PDF or calls experiment module (not provided).

The request models LoginBody, AssignmentBody and StartUploadBody are Pydantic classes rather than ordinary route functions. They validate incoming JSON types and bounds.

10. Complete Upload Request Lifecycle

• React calculates the file SHA-256 and sends POST /api/uploads/start with assignment ID, filename, size and hash.

• FastAPI validates the StartUploadBody, reads the logged-in user cookie and checks session-creation rate limit.

• upload_service.create_upload_session checks role, deadline and existing submissions, then stores a session and a chunk count.

• React divides the file into 512 KiB chunks and uploads them with PUT /api/uploads/{session_id}/chunks/{chunk_index}.

• FastAPI streams at most one chunk request body within its configured limit, then calls save_chunk and receive_chunk.

• receive_chunk checks authorization, size, index and expiry; it stores bytes and metadata and acknowledges success.

• If interrupted, React retrieves the upload status and sends only missing chunk indexes.

• After all chunks arrive, React calls POST /api/uploads/{session_id}/finalize.

• finalize_submission assembles chunks in index order, verifies SHA-256 and PDF properties, checks deadline policy, and stores the Submission record.

• receipt_view returns the final receipt and classification; authorized student or faculty may download the PDF.

11. Algorithms and Pseudocode

Sliding window rate limiting

```
recent = stored accepted timestamps for user
remove timestamps older than the window
if count(recent) >= limit: reject with retry_after
```

```
else: append current monotonic timestamp; allow
```

Missing-chunk recovery

```
received = set(chunk indexes stored for session)
missing = [i for i in range(total_chunks) if i not in received]
frontend uploads only indexes from missing
```

Finalization

```
if missing chunks: reject
for index in 0..total_chunks-1: append stored bytes to final PDF
verify assembled size and SHA-256 hash
verify PDF format and allowed deadline policy
create one Submission row; commit; issue receipt
```

12. Security Architecture and Known Limitations

• PBKDF2 password hashing, HTTP-only SameSite cookie, role checks and per-upload ownership checks are present.

• The in-memory SESSIONS dictionary is lost at restart; the token storage is not shared across workers and lacks an explicit token expiry in the uploaded auth.py.

• Origin checking in middleware is useful for this local demonstration but is not a complete production CSRF mitigation. HTTPS, secure cookies and a robust CSRF strategy would be needed for deployment.

• Local file/disk operations cannot be committed atomically with SQLite transactions. Power loss and process crashes can leave orphaned chunks or FINALIZING sessions.

• The four-transfer semaphore and RLock are process-local; running multiple workers does not enforce a common global limit.

• rate_limiter.reset() clears rate counter dictionaries without acquiring their locks.

• An intentionally rejected HTTP 429 is not a crashed server; compare it separately from HTTP 5xx and client timeouts.

• The comparison endpoint imports experiments.compare_limiters, which was not included in the latest supplied backend files; working experimental results are unverified.

13. Testing and Evaluation Plan

Test	Expected result
Valid account login	User cookie is issued; GET /api/auth/me returns identity.
New session after deadline	Server rejects creation with a clear error.
Two identical chunks	Second request is acknowledged as already received.
Changed duplicate chunk	Request rejected with conflict.
Interrupted upload	Reconnecting returns missing indexes and permits eligible resume.
Hash mismatch	Finalization rejected and not recorded as successful.
Repeated finalization	Same completed submission returned or no duplicate created.
Unauthorized download	Other student cannot access the PDF.

Test	Expected result
Baseline vs proposed traffic	Record attempts, completions, 429s, errors, retransmitted bytes and timings.

Recommended controlled experiment: use identical small local files and synthetic clients, document hardware and load, and test several concurrency levels. The test must measure effects rather than assume improvements. Do not claim the server handles hundreds of users until verified.

14. Senior-Mentor Viva: Questions and Answers

1. What makes DeadlineGuard more than Google Classroom?

We focus on separate session-admission and continuation limits with resumable transfers and deadline-specific recovery, then evaluate that behavior. We do not claim these concepts were invented here.

2. Why is a new upload session different from one chunk?

One session creates a logical upload attempt; many chunk requests transfer that same file. Applying the same strict counter to both can block legitimate continuation.

3. What is a 429 response?

HTTP Too Many Requests, an intentional admission decision, not proof of server failure.

4. Why use SHA-256?

To verify that the assembled file matches the committed expected bytes. It is not encryption or proof of authorship.

5. What does Depends(get_db) mean?

FastAPI invokes the database dependency, yields a session to the route, then runs cleanup afterward.

6. What is a transaction?

A group of database changes committed together; db.commit() persists them, while rollback undoes uncommitted changes.

7. What is idempotency?

A repeated operation has the same intended effect; identical duplicate chunks and repeated finalization are handled without producing duplicate submissions.

8. Why use a semaphore and a rate limiter?

A semaphore caps current simultaneous transfers; a rate limiter caps request frequency over a time window.

9. What if the server restarts?

SQLite metadata and disk chunks can survive, but in-memory login sessions and rate counters reset. The user needs to log in again.

10. What is the limitation of your experiment?

Only the comparison route is included in this upload, not the experiment implementation; no measured results can be claimed from these sources.

15. Running the Latest Backend

The README.txt included with the newer uploaded code specifies running from the project root:

```
.venv\Scripts\python.exe -m uvicorn main:app --app-dir backend --host 127.0.0.1 --port 8002
```

Health endpoint: http://127.0.0.1:8002/api/health. FastAPI interactive documentation: http://127.0.0.1:8002/docs. Note: this newer code uses top-level imports such as from database import ...; it is not the same package layout as the previously discussed backend.main app on port 8000.

The latest seed.py creates local demo users faculty1, student1 and student2 with a demo-only password. Never use those shared credentials for a real deployment.

16. Conclusion and Next Work

DeadlineGuard demonstrates meaningful server-side reasoning: decide whether a request is eligible, constrain admission, preserve transfer progress, verify reconstructed file integrity and record final submissions with timestamps. Its strongest practical demonstration is an interrupted upload that resumes and finishes under a clearly documented deadline policy.

Next steps: verify the missing experiment source and outputs; add automated recovery tests; improve expiration cleanup and crash recovery; secure authentication for deployment; and run repeatable, bounded load tests.

References

1. Chen, X., and Heidemann, J. (2005). “Flash Crowd Mitigation via Adaptive Admission Control Based on Application-Level Observations.” ACM Transactions on Internet Technology 5(3). https://doi.org/10.1145/1084772.1084776

2. Google Developers. “Resumable Media Uploads in the Google Data Protocol.” https://developers.google.com/gdata/docs/resumable_upload (technical documentation, not a research paper).

3. DeadlineGuard backend source files uploaded in this conversation (27 October 2026). Function descriptions and architecture are grounded in these supplied files.